

Metacognition: thinking about thinking

If you really want to learn, and you want to learn more quickly and more deeply, pay attention to how you pay attention. Think about how you think. Learn how you learn.

Most of us did not take courses on metacognition or learning theory when we were growing up. We were *expected* to learn, but rarely *taught* to learn.

But we assume that if you're holding this book, you really want to learn design patterns. And you probably don't want to spend a lot of time. And you want to *remember* what you read, and be able to apply it. And for that, you've got to *understand* it. To get the most from this book, or *any* book or learning experience, take responsibility for your brain. Your brain on *this* content.

The trick is to get your brain to see the new material you're learning as Really Important. Crucial to your well-being. As important as a tiger. Otherwise, you're in for a constant battle, with your brain doing its best to keep the new content from sticking.

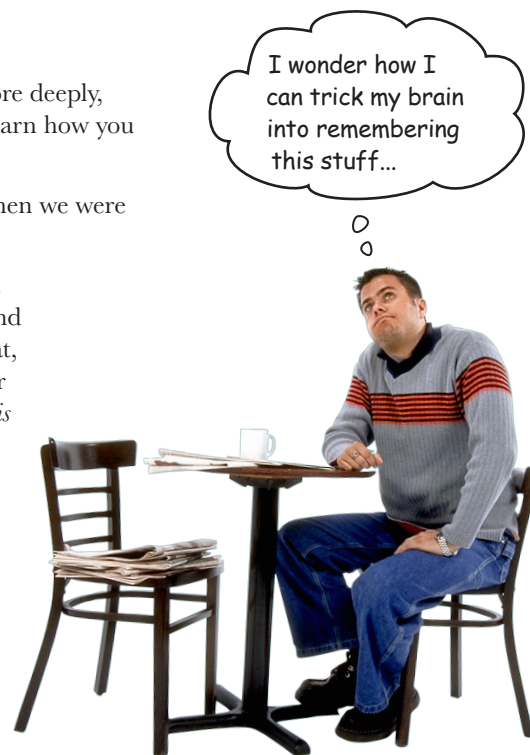
So how DO you get your brain to think Design Patterns are as important as a tiger?

There's the slow, tedious way, or the faster, more effective way. The slow way is about sheer repetition. You obviously know that you *are* able to learn and remember even the duller of topics, if you keep pounding on the same thing. With enough repetition, your brain says, "This doesn't *feel* important to him, but he keeps looking at the same thing *over and over and over*, so I suppose it must be."

The faster way is to do **anything that increases brain activity**, especially different *types* of brain activity. The things on the previous page are a big part of the solution, and they're all things that have been proven to help your brain work in your favor. For example, studies show that putting words *within* the pictures they describe (as opposed to somewhere else in the page, like a caption or in the body text) causes your brain to try to make sense of how the words and picture relate, and this causes more neurons to fire. More neurons firing = more chances for your brain to *get* that this is something worth paying attention to, and possibly recording.

A conversational style helps because people tend to pay more attention when they perceive that they're in a conversation, since they're expected to follow along and hold up their end. The amazing thing is, your brain doesn't necessarily *care* that the "conversation" is between you and a book! On the other hand, if the writing style is formal and dry, your brain perceives it the same way you experience being lectured to while sitting in a roomful of passive attendees. No need to stay awake.

But pictures and conversational style are just the beginning.



Here's what WE did:

We used **pictures**, because your brain is tuned for visuals, not text. As far as your brain's concerned, a picture really is worth 1,024 words. And when text and pictures work together, we embedded the text in the pictures because your brain works more effectively when the text is within the thing it refers to, as opposed to in a caption or buried somewhere else.

We used **redundancy**, saying the same thing in *different* ways and with different media types, and *multiple senses*, to increase the chance that the content gets coded into more than one area of your brain.

We used concepts and pictures in **unexpected** ways because your brain is tuned for novelty, and we used pictures and ideas with at least *some emotional content*, because your brain is tuned to pay attention to the biochemistry of emotions. That which causes you to *feel* something is more likely to be remembered, even if that feeling is nothing more than a little **humor, surprise, or interest**.

We used a personalized, **conversational style**, because your brain is tuned to pay more attention when it believes you're in a conversation than if it thinks you're passively listening to a presentation. Your brain does this even when you're *reading*.

We included more than 90 **activities**, because your brain is tuned to learn and remember more when you **do** things than when you read about things. And we made the exercises challenging-yet-do-able, because that's what most *people* prefer.

We used **multiple learning styles**, because *you* might prefer step-by-step procedures, while someone else wants to understand the big picture first, while someone else just wants to see a code example. But regardless of your own learning preference, *everyone* benefits from seeing the same content represented in multiple ways.

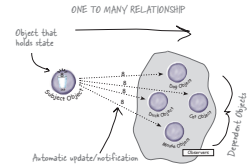
We included content for **both sides of your brain**, because the more of your brain you engage, the more likely you are to learn and remember, and the longer you can stay focused. Since working one side of the brain often means giving the other side a chance to rest, you can be more productive at learning for a longer period of time.

And we included **stories** and exercises that present **more than one point of view**, because your brain is tuned to learn more deeply when it's forced to make evaluations and judgments.

We included **challenges**, both with exercises and by asking **questions** that don't always have a straight answer, because your brain is tuned to learn and remember when it has to *work* at something. Think about it—you can't get your *body* in shape just by *watching* people at the gym. But we did our best to make sure that when you're working hard, it's on the *right* things. That **you're not spending one extra dendrite** processing a hard-to-understand example, or parsing difficult, jargon-laden, or overly terse text.

We used **people**. In stories, examples, pictures, etc., because, well, because *you're* a person. And your brain pays more attention to *people* than it does to *things*.

We used an **80/20** approach. We assume that if you're going for a PhD in software design, this won't be your only book. So we don't talk about *everything*. Just the stuff you'll actually *need*.



The Patterns Guru

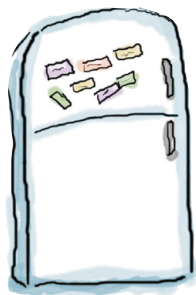


BULLET POINTS



Puzzles





Cut this out and stick it on your refrigerator.

Here's what YOU can do to bend your brain into submission

So, we did our part. The rest is up to you. These tips are a starting point; listen to your brain and figure out what works for you and what doesn't. Try new things.

1 **Slow down. The more you understand, the less you have to memorize.**

Don't just *read*. Stop and think. When the book asks you a question, don't just skip to the answer. Imagine that someone really is asking the question. The more deeply you force your brain to think, the better chance you have of learning and remembering.

2 **Do the exercises. Write your own notes.**

We put them in, but if we did them for you, that would be like having someone else do your workouts for you. And don't just *look* at the exercises. **Use a pencil.** There's plenty of evidence that physical activity *while* learning can increase the learning.

3 **Read the "There Are No Dumb Questions"**

That means all of them. They're not optional sidebars—*they're part of the core content!* Don't skip them.

4 **Make this the last thing you read before bed. Or at least the last *challenging* thing.**

Part of the learning (especially the transfer to long-term memory) happens *after* you put the book down. Your brain needs time on its own, to do more processing. If you put in something new during that processing time, some of what you just learned will be lost.

5 **Drink water. Lots of it.**

Your brain works best in a nice bath of fluid. Dehydration (which can happen before you ever feel thirsty) decreases cognitive function.

6 **Talk about it. Out loud.**

Speaking activates a different part of the brain. If you're trying to understand something, or increase your chance of remembering it later, say it out loud. Better still, try to explain it out loud to someone else. You'll learn more quickly, and you might uncover ideas you hadn't known were there when you were reading about it.

7 **Listen to your brain.**

Pay attention to whether your brain is getting overloaded. If you find yourself starting to skim the surface or forget what you just read, it's time for a break. Once you go past a certain point, you won't learn faster by trying to shove more in, and you might even hurt the process.

8 **Feel something!**

Your brain needs to know that this *matters*. Get involved with the stories. Make up your own captions for the photos. Groaning over a bad joke is *still* better than feeling nothing at all.

9 **Design something!**

Apply this to something new you're designing, or refactor an older project. Just *do something* to get some experience beyond the exercises and activities in this book. All you need is a pencil and a problem to solve...a problem that might benefit from one or more design patterns.

Read Me

This is a learning experience, not a reference book. We deliberately stripped out everything that might get in the way of learning whatever it is we’re working on at that point in the book. And the first time through, you need to begin at the beginning, because the book makes assumptions about what you’ve already seen and learned.

We use simple UML-like diagrams.

Although there’s a good chance you’ve run across UML, it’s not covered in the book, and it’s not a prerequisite for the book. If you’ve never seen UML before, don’t worry, we’ll give you a few pointers along the way. So in other words, you won’t have to worry about Design Patterns and UML at the same time. Our diagrams are “UML-like”—while we try to be true to UML there are times we bend the rules a bit, usually for our own selfish artistic reasons.

We don’t cover every single Design Pattern ever created.

There are a *lot* of Design Patterns: the original foundational patterns (known as the GoF patterns), enterprise Java patterns, architectural patterns, game design patterns, and a lot more. But our goal was to make sure the book weighed less than the person reading it, so we don’t cover them all here. Our focus is on the core patterns that *matter* from the original GoF object-oriented patterns, and making sure that you really, truly, deeply understand how and when to use them. You will find a brief look at some of the other patterns (the ones you’re far less likely to use) in the appendix. In any case, once you’re done with *Head First Design Patterns*, you’ll be able to pick up any patterns catalog and get up to speed quickly.

The activities are NOT optional.

The exercises and activities are not add-ons; they’re part of the core content of the book. Some of them are to help with memory, some for understanding, and some to help you apply what you’ve learned. ***Don’t skip the exercises.*** The crossword puzzles are the only things you don’t *have* to do, but they’re good for giving your brain a chance to think about the words from a different context.

We use the word “composition” in the general OO sense, which is more flexible than the strict UML use of “composition.”

When we say “one object is composed with another object” we mean that they are related by a HAS-A relationship. Our use reflects the traditional use of the term and is the one used in the GoF text (you’ll learn what that is later). More recently, UML has refined this term into several types of composition. If you are an UML expert, you’ll still be able to read the book and you should be able to easily map the use of composition to more refined terms as you read.

We use a simpler, modified version of UML. ↴

Director
getMovies getOscars() getKevinBaconDegrees()

The redundancy is intentional and important.

One distinct difference in a Head First book is that we want you to *really* get it. And we want you to finish the book remembering what you've learned. Most reference books don't have retention and recall as a goal, but this book is about *learning*, so you'll see some of the same concepts come up more than once.

The code examples are as lean as possible.

Our readers tell us that it's frustrating to wade through 200 lines of code looking for the two lines they need to understand. Most examples in this book are shown within the smallest possible context, so that the part you're trying to learn is clear and simple. Don't expect all of the code to be robust, or even complete—the examples are written specifically for learning, and aren't always fully functional.

In some cases, we haven't included all of the import statements needed, but we assume that if you're a Java programmer, you know that `ArrayList` is in `java.util`, for example. If the imports were not part of the normal core JSE API, we mention it. We've also placed all the source code on the web so you can download it. You'll find it at <http://wickedlysmart.com/head-first-design-patterns>.

Also, for the sake of focusing on the learning side of the code, we did not put our classes into packages (in other words, they're all in the Java default package). We don't recommend this in the real world, and when you download the code examples from this book, you'll find that all classes *are* in packages.

The Brain Power exercises don't have answers.

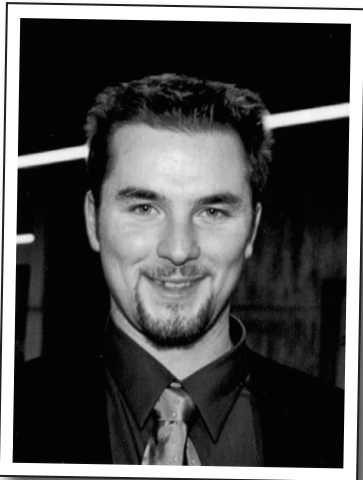
For some of them, there is no right answer, and for others, part of the learning experience of the Brain Power activities is for you to decide if and when your answers are right. In some of the Brain Power exercises, you will find hints to point you in the right direction.

Tech Reviewers

Jef Cumps



Valentin Crettaz



Barney Marispini



Ike Van Atta



Fearless leader of
the HFDP Extreme
Review Team

Johannes deJong

In memory of Philippe Maquet, 1960 – 2004. Your amazing technical expertise, relentless enthusiasm, and deep concern for the learner will inspire us always.



Philippe Maquet



Dirk Schreckmann



Mark Spritzler



Jason Menard

Tech Reviewers, 2nd Edition



Julian Setiawan



George Heineman

↖ 2nd Edition Reviewer MVP!



David Powers



Trisha Gee

Acknowledgments

From the first edition

At O'Reilly:

Our biggest thanks to **Mike Loukides** at O'Reilly, for starting it all and helping to shape the Head First concept into a series. And a big thanks to the driving force behind Head First, **Tim O'Reilly**. Thanks to the clever Head First “series mom” **Kyle Hart**, “In Design King” **Ron Bilodeau**, rock-and-roll star **Ellie Volkhausen** for her inspired cover design, **Melanie Yarbrough** for shepherding production, **Colleen Gorman** and **Rachel Monaghan** for their hardcore copyedits, and **Bob Pfahler** for a much improved index. Finally, thanks to **Mike Hendrickson** and **Meghan Blanchette** for championing this book and building the team.

Our intrepid reviewers:

We are extremely grateful for our technical review director **Johannes deJong**. You are our hero, Johannes. And we deeply appreciate the contributions of the co-manager of the **Javaranch** review team, the late **Philippe Maquet**. You have single-handedly brightened the lives of thousands of developers, and the impact you've had on their (and our) lives is forever. **Jef Cumps** is scarily good at finding problems in our draft chapters, and once again made a huge difference for the book. Thanks Jef! **Valentin Cretazz** (AOP guy), who has been with us from the very first Head First book, proved (as always) just how much we really need his technical expertise and insight. You rock Valentin (but lose the tie).

Two newcomers to the HF review team, **Barney Marispini** and **Ike Van Atta**, did a kick-butt job on the book—you guys gave us some *really* crucial feedback. Thanks for joining the team.

We also got some excellent technical help from Javaranch moderators/gurus **Mark Spritzler**, **Jason Menard**, **Dirk Schreckmann**, **Thomas Paul**, and **Margarita Isaeva**. And as always, thanks especially to the javaranch.com Trail Boss, **Paul Wheaton**.

Thanks to the finalists of the Javaranch “Pick the *Head First Design Patterns* Cover” contest. The winner, Si Brewster, submitted the winning essay that persuaded us to pick the woman you see on our cover. Other finalists include Andrew Esse, Gian Franco Casula, Helen Crosbie, Pho Tek, Helen Thomas, Sateesh Kommineni, and Jeff Fisher.

For the 2014 update to the book, we are so grateful to the following technical reviewers: George Hoffer, Ted Hill, Todd Bartoszkiewicz, Sylvain Tenier, Scott Davidson, Kevin Ryan, Rich Ward, Mark Francis Jaeger, Mark Masse, Glenn Ray, Bayard Fetler, Paul Higgins, Matt Carpenter, Julia Williams, Matt McCullough, and Mary Ann Belarmino.

Acknowledgments

From the second edition

At O'Reilly:

First and foremost, **Mary Treseler** is the superpower who makes everything happen and we are eternally grateful to her for all she does for O'Reilly, Head First, and the authors. **Melissa Duffield** and **Michele Cronin** cleared many paths that made this second edition happen. **Rachel Monaghan** did an amazing copy edit, giving a new sparkle to our text. **Kristen Brown** made the book look beautiful online and in print. **Ellie Volckhausen** worked her magic and designed a brilliant new cover for the second edition. Thank you all!

Our 2nd edition reviewers:

We're grateful to our 2nd edition technical reviewers for picking up the task 15 years later: **David Powers** is our go-to reviewer (he's ours, don't even think about asking him to review *your* book) because he doesn't miss a thing. **George Heineman** went above and beyond with his detailed comments, suggestions, and feedback, and he received this edition's technical MVP award. **Trisha Gee** and **Julian Setiawan** provided the invaluable Java savvy we needed to help us avoid those embarrassing and cringe-worthy Java mistakes. Thank you all!

Very Special Thanks

A very special thanks to **Erich Gamma**, who went far beyond the call of duty in reviewing this book (he even took a draft with him on vacation). Erich, your interest in this book inspired us, and your thorough technical review improved it immeasurably. Thanks as well to the entire **Gang of Four** for their support and interest, and for making a special appearance in Objectville. We are also indebted to **Ward Cunningham** and the patterns community who created the Portland Pattern Repository—an indispensable resource for us in writing this book.

A big thank you to **Mike Loukides**, **Mike Hendrickson**, and **Meghan Blanchette**. Mike L. was with us every step of the way. Mike, your insightful feedback helped shape the book, and your encouragement kept us moving ahead. Mike H., thanks for your persistence over five years in trying to get us to write a patterns book; we finally did it and we're glad we waited for Head First.

It takes a village to write a technical book: **Bill Pugh** and **Ken Arnold** gave us expert advice on Singleton. **Joshua Marinacci** provided rockin' Swing tips and advice. **John Brewer's** "Why a Duck?" paper inspired SimUDuck (and we're glad he likes ducks too). **Dan Friedman** inspired the Little Singleton example. **Daniel Steinberg** acted as our "technical liason" and our emotional support network. Thanks to Apple's **James Dempsey** for allowing us to use his MVC song. And thank you to **Richard Warburton**, who made sure our Java 8 code updates were up to snuff for this updated edition of the book.

Last, a personal thank you to the **Javaranch review team** for their top-notch reviews and warm support. There's more of you in this book than you know.

Writing a Head First book is a wild ride with two amazing tour guides: **Kathy Sierra** and **Bert Bates**. With Kathy and Bert you throw out all book writing convention and enter a world full of storytelling, learning theory, cognitive science, and pop culture, where the reader always rules.

1 intro to Design Patterns

Welcome to Design Patterns

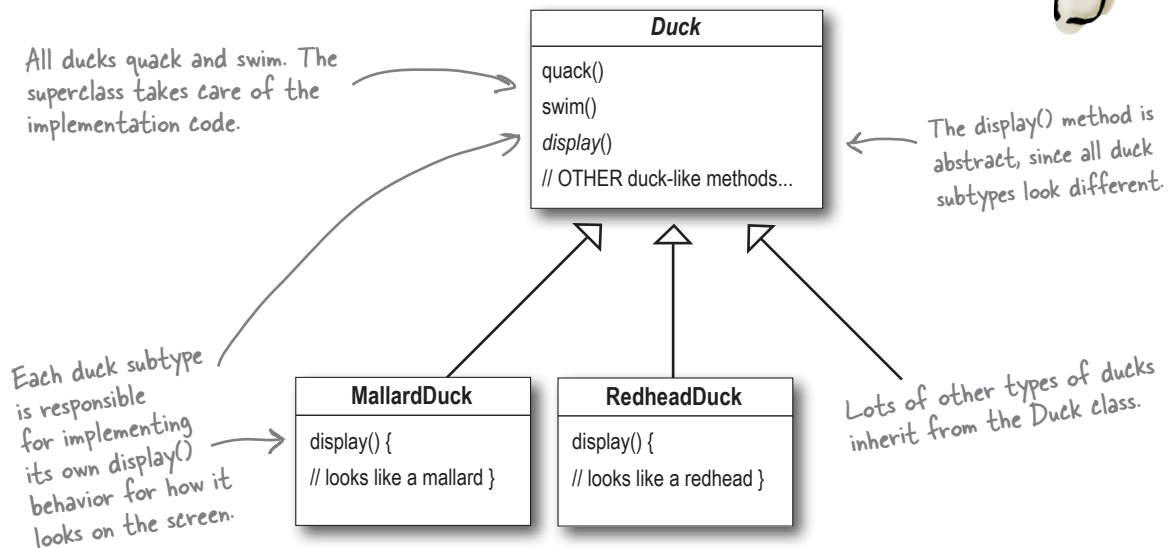
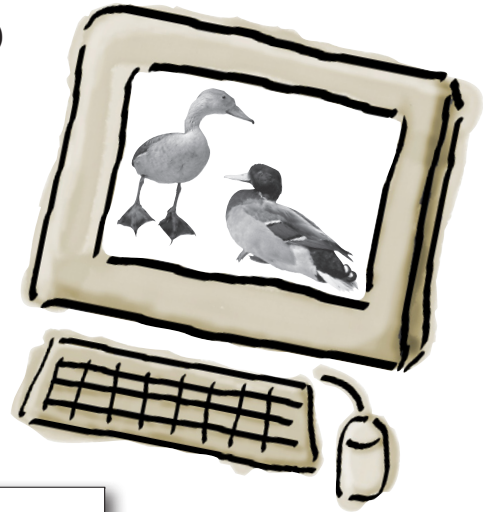


Now that we're living in Objectville, we've just got to get into Design Patterns...everyone is doing them. Soon we'll be the hit of Jim and Betty's Wednesday night patterns group!

Someone has already solved your problems. In this chapter, you'll learn why (and how) you can exploit the wisdom and lessons learned by other developers who've been down the same design problem road and survived the trip. Before we're done, we'll look at the use and benefits of design patterns, look at some key object-oriented (OO) design principles, and walk through an example of how one pattern works. The best way to use patterns is to *load your brain* with them and then *recognize places* in your designs and existing applications where you can *apply them*. Instead of *code* reuse, with patterns you get *experience* reuse.

It started with a simple SimUDuck app

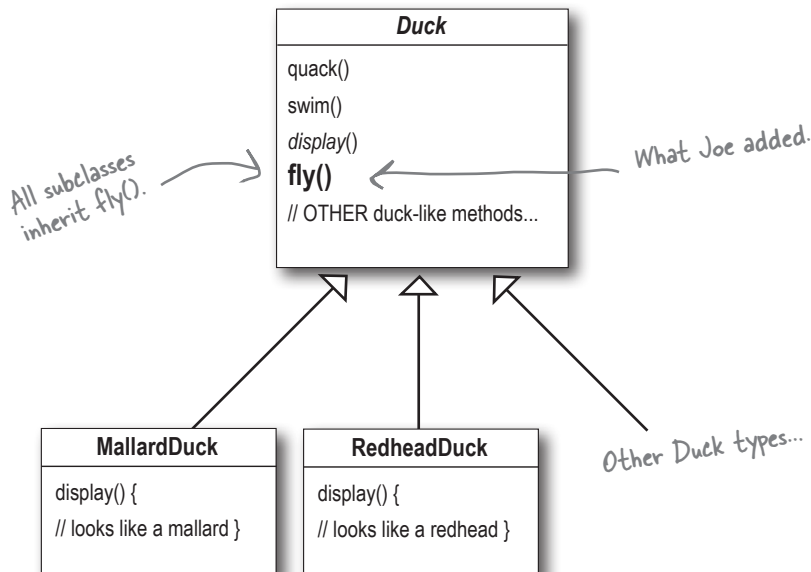
Joe works for a company that makes a highly successful duck pond simulation game, *SimUDuck*. The game can show a large variety of duck species swimming and making quacking sounds. The initial designers of the system used standard OO techniques and created one Duck superclass from which all other duck types inherit.



In the last year, the company has been under increasing pressure from competitors. After a week-long off-site brainstorming session over golf, the company executives think it's time for a big innovation. They need something *really* impressive to show at the upcoming shareholders meeting in Maui *next week*.

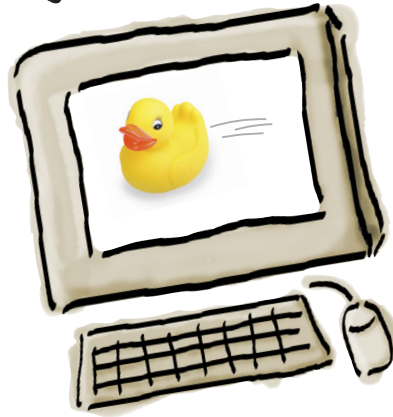
But now we need the ducks to FLY

The executives decided that flying ducks is just what the simulator needs to blow away the competitors. And of course Joe's manager told them it'll be no problem for Joe to just whip something up in a week. "After all," said Joe's boss, "he's an OO programmer...*how hard can it be?*"



But something went horribly wrong...

Joe, I'm at the shareholders meeting. They just gave a demo and there were **rubber duckies** flying around the screen. Was this your idea of a joke?



What happened?

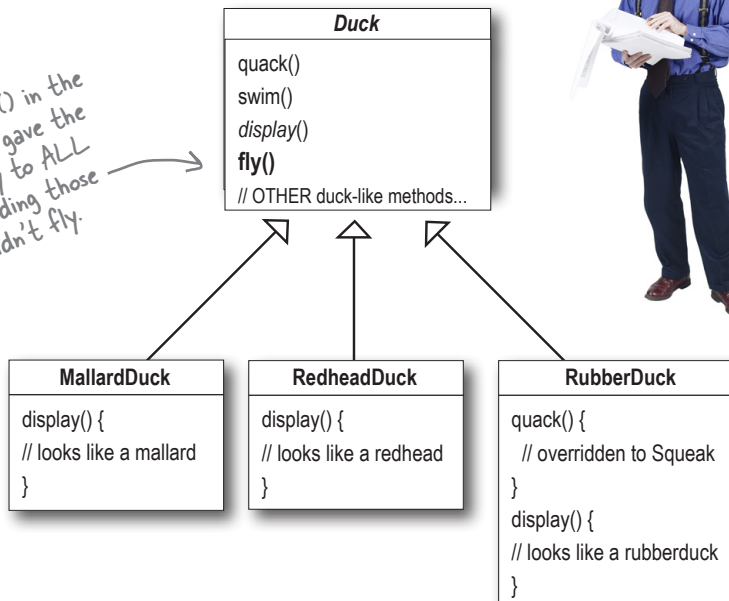
Joe failed to notice that not *all* subclasses of Duck should *fly*. When Joe added new behavior to the Duck superclass, he was also adding behavior that was *not* appropriate for some Duck subclasses. He now has flying inanimate objects in the SimUDuck program.

A localized update to the code caused a non-local side effect (flying rubber ducks)!

Okay, so there's a slight flaw in my design. I don't see why they can't just call it a "feature." It's kind of cute...

What Joe thought was a great use of inheritance for the purpose of reuse hasn't turned out so well when it comes to maintenance.

By putting `fly()` in the superclass, he gave the flying ability to *ALL* ducks, including those that shouldn't fly.



Notice too, that rubber ducks don't quack, so `quack()` is overridden to "Squeak".

Joe thinks about inheritance...

I could always just override the fly() method in rubber duck, like I have with the quack() method...



RubberDuck
quack() { // squeak }
display() { // rubber duck }
fly() { // override to do nothing }

But then what happens when we add wooden decoy ducks to the program? They aren't supposed to fly or quack...



DecoyDuck
quack() { // override to do nothing }
display() { // decoy duck }
fly() { // override to do nothing }

Here's another class in the hierarchy; notice that like RubberDuck, it doesn't fly, but it also doesn't quack.

Sharpen your pencil



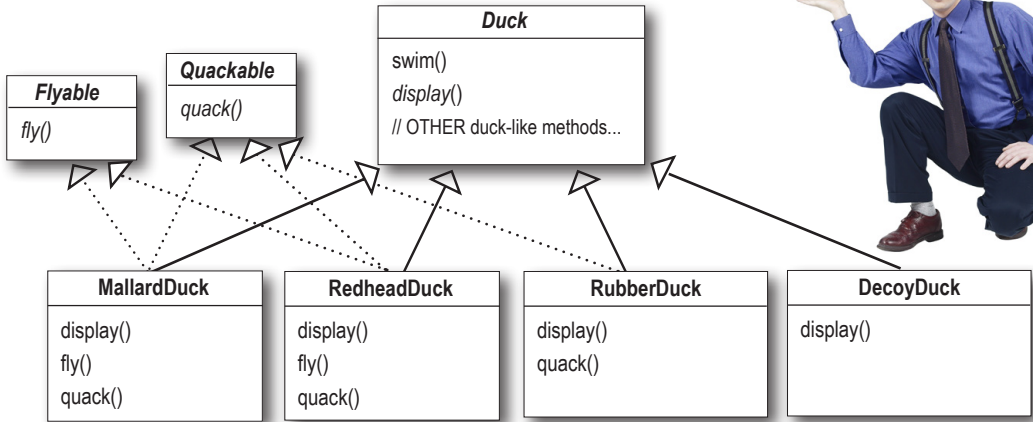
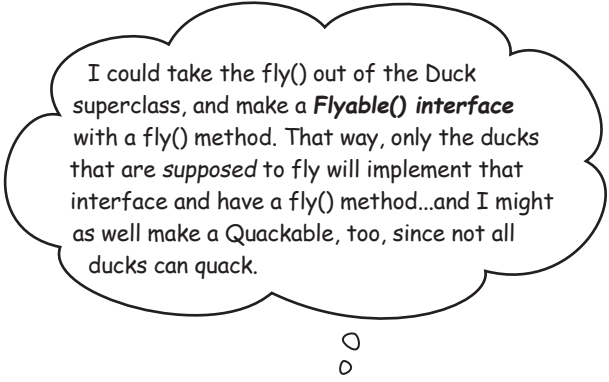
Which of the following are disadvantages of using *inheritance* to provide Duck behavior? (Choose all that apply.)

- ☐ A. Code is duplicated across subclasses.
- ☐ B. Runtime behavior changes are difficult.
- ☐ C. We can't make ducks dance.
- ☐ D. It's hard to gain knowledge of all duck behaviors.
- ☐ E. Ducks can't fly and quack at the same time.
- ☐ F. Changes can unintentionally affect other ducks.

How about an interface?

Joe realized that inheritance probably wasn't the answer, because he just got a memo that says that the executives now want to update the product every six months (in ways they haven't yet decided on). Joe knows the spec will keep changing and he'll be forced to look at and possibly override fly() and quack() for every new Duck subclass that's ever added to the program....*forever*.

So, he needs a cleaner way to have only *some* (but not *all*) of the duck types fly or quack.



What do YOU think about this design?

That is, like, the dumbest idea you've come up with. **Can you say, "duplicate code"?** If you thought having to override a few methods was bad, how are you gonna feel when you need to make a little change to the flying behavior...*in all 48 of the flying Duck subclasses?!*



What would you do if you were Joe?

We know that not *all* of the subclasses should have flying or quacking behavior, so inheritance isn't the right answer. But while having the subclasses implement Flyable and/or Quackable solves *part* of the problem (no inappropriately flying rubber ducks), it completely destroys code reuse for those behaviors, so it just creates a *different* maintenance nightmare. And of course there might be more than one kind of flying behavior even among the ducks that *do* fly...

At this point you might be waiting for a Design Pattern to come riding in on a white horse and save the day. But what fun would that be? No, we're going to figure out a solution the old-fashioned way—*by applying good OO software design principles.*

Wouldn't it be dreamy if there were a way to build software so that when we need to change it, we could do so with the least possible impact on the existing code? We could spend less time reworking code and more making the program do cooler things...



The one constant in software development

Okay, what’s the one thing you can always count on in software development?

No matter where you work, what you’re building, or what language you are programming in, what’s the one true constant that will be with you always?

CHANGE

(use a mirror to see the answer)

No matter how well you design an application, over time an application must grow and change or it will *die*.

Sharpen your pencil

Lots of things can drive change. List some reasons you’ve had to change code in your applications (we put in a couple of our own to get you started). Check your answers with the solution at the end of the chapter before you go on.

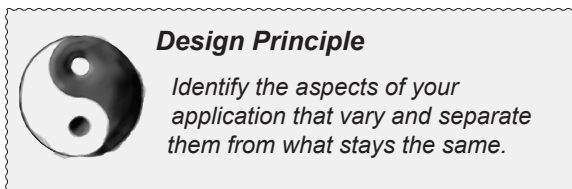
My customers or users decide they want something else, or they want new functionality.

My company decided it is going with another database vendor and it is also purchasing its data from another supplier that uses a different data format. Argh!

Zeroing in on the problem...

So we know using inheritance hasn't worked out very well, since the duck behavior keeps changing across the subclasses, and it's not appropriate for *all* subclasses to have those behaviors. The Flyable and Quackable interface sounded promising at first—only ducks that really do fly will be Flyable, etc.—except Java interfaces typically have no implementation code, so no code reuse. In either case, whenever you need to modify a behavior, you're often forced to track down and change it in all the different subclasses where that behavior is defined, probably introducing *new* bugs along the way!

Luckily, there's a design principle for just this situation.



↪ The first of many design principles. We'll spend more time on these throughout the book.

In other words, if you've got some aspect of your code that is changing, say with every new requirement, then you know you've got a behavior that needs to be pulled out and separated from all the stuff that doesn't change.

Here's another way to think about this principle: ***take the parts that vary and encapsulate them, so that later you can alter or extend the parts that vary without affecting those that don't.***

As simple as this concept is, it forms the basis for almost every design pattern. All patterns provide a way to let *some part of a system vary independently of all other parts.*

Okay, time to pull the duck behavior out of the Duck classes!

Take what varies and "encapsulate" it so it won't affect the rest of your code.

The result? Fewer unintended consequences from code changes and more flexibility in your systems!

Separating what changes from what stays the same

Where do we start? As far as we can tell, other than the problems with `fly()` and `quack()`, the Duck class is working well and there are no other parts of it that appear to vary or change frequently. So, other than a few slight changes, we're going to pretty much leave the Duck class alone.

Now, to separate the “parts that change from those that stay the same,” we are going to create two *sets* of classes (totally apart from Duck), one for *fly* and one for *quack*. Each set of classes will hold all the implementations of the respective behavior. For instance, we might have *one* class that implements *quacking*, *another* that implements *squeaking*, and *another* that implements *silence*.

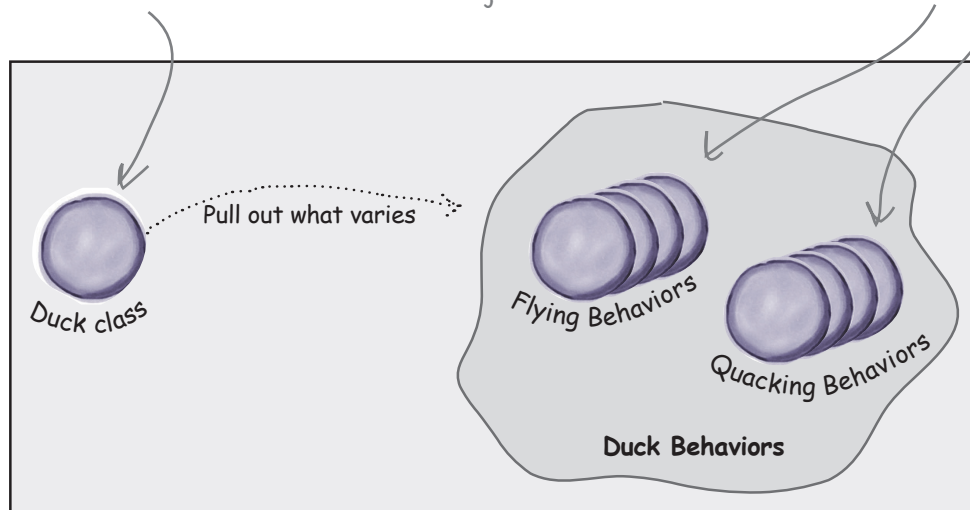
We know that `fly()` and `quack()` are the parts of the Duck class that vary across ducks.

To separate these behaviors from the Duck class, we'll pull both methods out of the Duck class and create a new set of classes to represent each behavior.

The Duck class is still the superclass of all ducks, but we are pulling out the fly and quack behaviors and putting them into another class structure.

Now flying and quacking each get their own set of classes.

Various behavior implementations are going to live here.



Designing the Duck Behaviors

So how are we going to design the set of classes that implement the fly and quack behaviors?

We'd like to keep things flexible; after all, it was the inflexibility in the duck behaviors that got us into trouble in the first place. And we know that we want to *assign* behaviors to the instances of Duck. For example, we might want to instantiate a new MallardDuck instance and initialize it with a specific *type* of flying behavior. And while we're there, why not make sure that we can change the behavior of a duck dynamically? In other words, we should include behavior setter methods in the Duck classes so that we can *change* the MallardDuck's flying behavior *at runtime*.

Given these goals, let's look at our second design principle:



We'll use an interface to represent each behavior—for instance, FlyBehavior and QuackBehavior—and each implementation of a *behavior* will implement one of those interfaces.

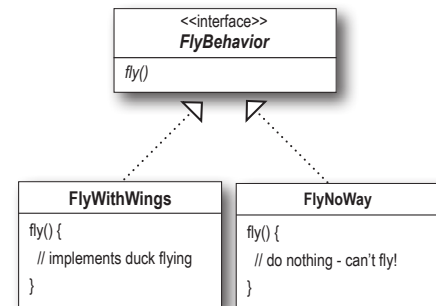
So this time it won't be the *Duck* classes that will implement the flying and quacking interfaces. Instead, we'll make a set of classes whose entire reason for living is to represent a behavior (for example, “squeaking”), and it's the *behavior* class, rather than the Duck class, that will implement the behavior interface.

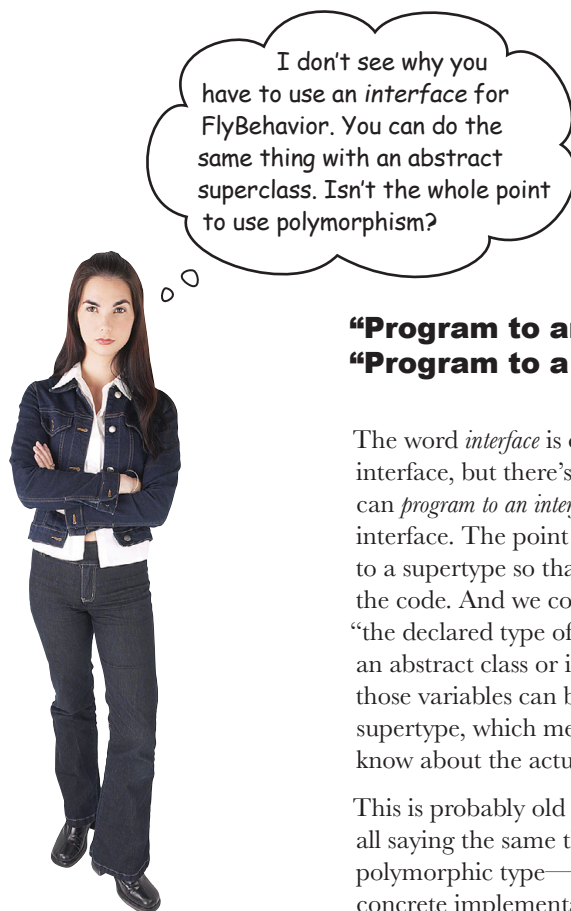
This is in contrast to the way we were doing things before, where a behavior came either from a concrete implementation in the superclass Duck, or by providing a specialized implementation in the subclass itself. In both cases we were relying on an *implementation*. We were locked into using that specific implementation and there was no room for changing the behavior (other than writing more code).

With our new design, the Duck subclasses will use a behavior represented by an *interface* (FlyBehavior and QuackBehavior), so that the actual *implementation* of the behavior (in other words, the specific concrete behavior coded in the class that implements the FlyBehavior or QuackBehavior) won't be locked into the Duck subclass.

From now on, the Duck behaviors will live in a separate class—a class that implements a particular behavior interface.

That way, the Duck classes won't need to know any of the implementation details for their own behaviors.





“Program to an interface” really means “Program to a supertype.”

The word *interface* is overloaded here. There’s the *concept* of an interface, but there’s also the Java *construct* of an interface. You can *program to an interface* without having to actually use a Java interface. The point is to exploit polymorphism by programming to a supertype so that the actual runtime object isn’t locked into the code. And we could rephrase “program to a supertype” as “the declared type of the variables should be a supertype, usually an abstract class or interface, so that the objects assigned to those variables can be of any concrete implementation of the supertype, which means the class declaring them doesn’t have to know about the actual object types!”

This is probably old news to you, but just to make sure we’re all saying the same thing, here’s a simple example of using a polymorphic type—imagine an abstract class `Animal`, with two concrete implementations, `Dog` and `Cat`.

Programming to an implementation would be:

```
Dog d = new Dog();
d.bark();
```

Declaring the variable “d” as type `Dog` (a concrete implementation of `Animal`) forces us to code to a concrete implementation.

But **programming to an interface/supertype** would be:

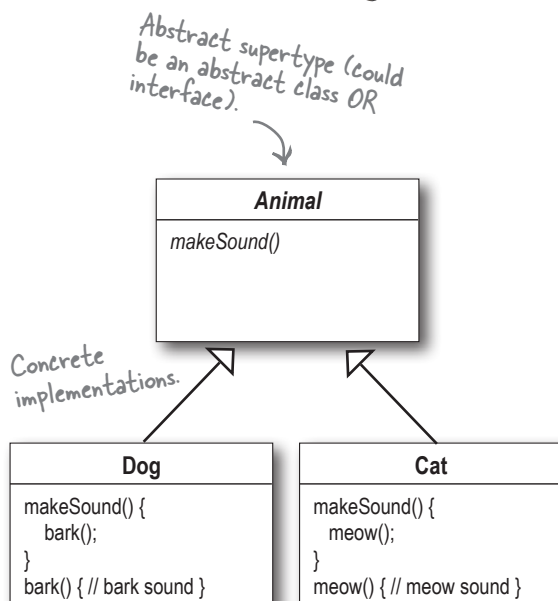
```
Animal animal = new Dog();
animal.makeSound();
```

We know it’s a `Dog`, but we can now use the `animal` reference polymorphically.

Even better, rather than hardcoding the instantiation of the subtype (like `new Dog()`) into the code, **assign the concrete implementation object at runtime**:

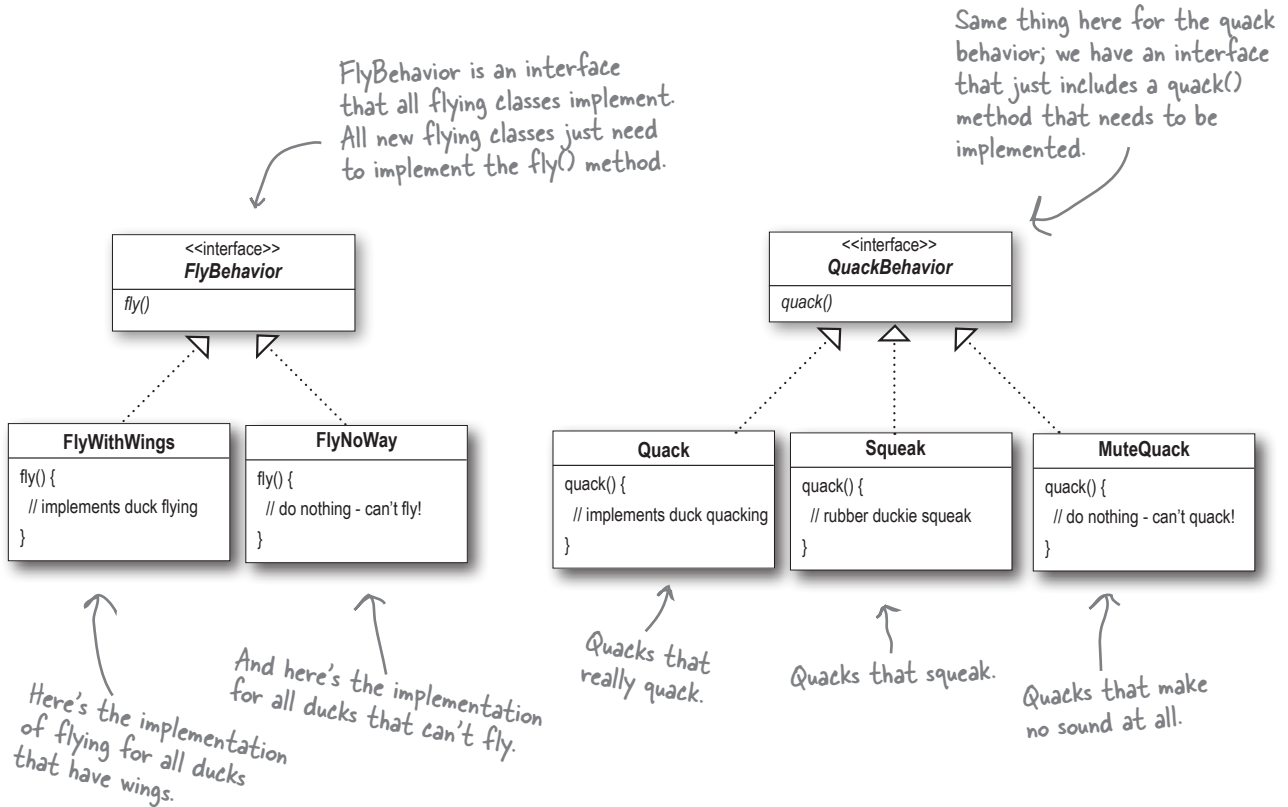
```
a = getAnimal();
a.makeSound();
```

We don’t know *WHAT* the actual `animal` subtype is...all we care about is that it knows how to respond to `makeSound()`.



Implementing the Duck Behaviors

Here we have the two interfaces, FlyBehavior and QuackBehavior, along with the corresponding classes that implement each concrete behavior:



With this design, other types of objects can reuse our fly and quack behaviors because these behaviors are no longer hidden away in our Duck classes!

And we can add new behaviors without modifying any of our existing behavior classes or touching any of the Duck classes that use flying behaviors.

So we get the benefit of **REUSE** without all the baggage that comes along with inheritance.

there are no
Dumb Questions

Q: Do I always have to implement my application first, see where things are changing, and then go back to separate and encapsulate those things?

A: Not always; often when you are designing an application, you anticipate those areas that are going to vary and then go ahead and build the flexibility to deal with it into your code. You'll find that the principles and patterns can be applied at any stage of the development lifecycle.

Q: Should we make Duck an interface too?

A: Not in this case. As you'll see once we've got everything hooked together, we do benefit by having Duck not be an interface, and having specific ducks, like MallardDuck, inherit common properties and methods. Now that we've removed what varies from the Duck inheritance, we get the benefits of this structure without the problems.

Q: It feels a little weird to have a class that's just a behavior. Aren't classes supposed to represent things? Aren't classes supposed to have both state AND behavior?

A: In an OO system, yes, classes represent things that generally have both state (instance variables) and methods. And in this case, the thing happens to be a behavior. But even a behavior can still have state and methods; a flying behavior might have instance variables representing the attributes for the flying (wing beats per minute, max altitude, speed, etc.) behavior.

 Sharpen your pencil

- 1 Using our new design, what would you do if you needed to add rocket-powered flying to the SimUDuck app?
- 2 Can you think of a class that might want to use the Quack behavior that isn't a duck?

1) Create a FlyRocketPowered class that implements the FlyBehavior interface.
2) One example, a duck call (a device that makes duck sounds).

Answers:

Integrating the Duck Behaviors

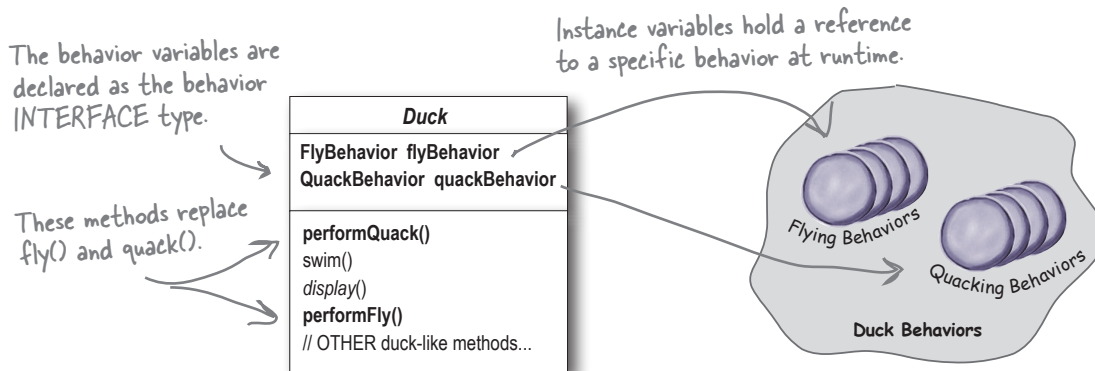
Here's the key: A Duck will now delegate its flying and quacking behaviors, instead of using quacking and flying methods defined in the Duck class (or subclass).

Here's how:

- 1 First we'll add two instance variables of type **FlyBehavior** and **QuackBehavior**—let's call them `flyBehavior` and `quackBehavior`. Each concrete duck object will assign to those variables a *specific* behavior at runtime, like `FlyWithWings` for flying and `Squeak` for quacking.

We'll also remove the `fly()` and `quack()` methods from the `Duck` class (and any subclasses) because we've moved this behavior out into the `FlyBehavior` and `QuackBehavior` classes.

We'll replace `fly()` and `quack()` in the `Duck` class with two similar methods, called `performFly()` and `performQuack()`; you'll see how they work next.



- 2 Now we implement `performQuack()`:

```

public abstract class Duck {
    QuackBehavior quackBehavior;
    // more

    public void performQuack() {
        quackBehavior.quack();
    }
}

```

Each Duck has a reference to something that implements the `QuackBehavior` interface.

Rather than handling the quack behavior itself, the `Duck` object delegates that behavior to the object referenced by `quackBehavior`.

Pretty simple, huh? To perform the quack, a `Duck` just asks the object that is referenced by `quackBehavior` to quack for it. In this part of the code we don't care *what kind of* object the concrete `Duck` is, ***all we care about is that it knows how to quack()***!

More integration...

- 3 Okay, time to worry about **how** the flyBehavior and quackBehavior instance variables **are set**. Let's take a look at the MallardDuck class:

```
public class MallardDuck extends Duck {
```

```
    public MallardDuck() {
        quackBehavior = new Quack();
        flyBehavior = new FlyWithWings();
    }
```

Remember, MallardDuck inherits the quackBehavior and flyBehavior instance variables from class Duck.

A MallardDuck uses the Quack class to handle its quack, so when performQuack() is called, the responsibility for the quack is delegated to the Quack object and we get a real quack.

And it uses FlyWithWings as its FlyBehavior type.

```
    public void display() {
        System.out.println("I'm a real Mallard duck");
    }
}
```

MallardDuck's quack is a real live duck **quack**, not a **squeak** and not a **mute quack**. When a MallardDuck is instantiated, its constructor initializes the MallardDuck's inherited quackBehavior instance variable to a new instance of type Quack (a QuackBehavior concrete implementation class).

And the same is true for the duck's flying behavior—the MallardDuck's constructor initializes the inherited flyBehavior instance variable with an instance of type FlyWithWings (a FlyBehavior concrete implementation class).



Wait a second, didn't you say we should NOT program to an implementation? But what are we doing in that constructor? We're making a new instance of a concrete Quack implementation class!

Good catch, that's exactly what we're doing... *for now.*

Later in the book we'll have more patterns in our toolbox that can help us fix it.

Still, notice that while we *are* setting the behaviors to concrete classes (by instantiating a behavior class like Quack or FlyWithWings and assigning it to our behavior reference variable), we could *easily* change that at runtime.

So, we still have a lot of flexibility here. That said, we're doing a poor job of initializing the instance variables in a flexible way. But think about it: since the quackBehavior instance variable is an interface type, we could (through the magic of polymorphism) dynamically assign a different QuackBehavior implementation class at runtime.

Take a moment and think about how you would implement a duck so that its behavior could change at runtime. (You'll see the code that does this a few pages from now.)

Testing the Duck code

- 1 Type and compile the Duck class below (Duck.java), and the MallardDuck class from two pages back (MallardDuck.java).

```
public abstract class Duck {

    FlyBehavior flyBehavior;
    QuackBehavior quackBehavior;
    public Duck() { }

    public abstract void display();

    public void performFly() {
        flyBehavior.fly();
    }

    public void performQuack() {
        quackBehavior.quack();
    }

    public void swim() {
        System.out.println("All ducks float, even decoys!");
    }
}
```

Declare two reference variables for the behavior interface types. All duck subclasses (in the same package) inherit these.

Delegate to the behavior class.

- 2 Type and compile the FlyBehavior interface (FlyBehavior.java) and the two behavior implementation classes (FlyWithWings.java and FlyNoWay.java).

```
public interface FlyBehavior {
    public void fly();
}
```

```
public class FlyWithWings implements FlyBehavior {
    public void fly() {
        System.out.println("I'm flying!!");
    }
}
```

```
public class FlyNoWay implements FlyBehavior {
    public void fly() {
        System.out.println("I can't fly");
    }
}
```

The interface that all flying behavior classes implement.

Flying behavior implementation for ducks that DO fly...

Flying behavior implementation for ducks that do NOT fly (like rubber ducks and decoy ducks).

Testing the Duck code, continued...

- ③ **Type and compile the QuackBehavior interface (QuackBehavior.java) and the three behavior implementation classes (Quack.java, MuteQuack.java, and Squeak.java).**

```
public interface QuackBehavior {
    public void quack();
}

public class Quack implements QuackBehavior {
    public void quack() {
        System.out.println("Quack");
    }
}

public class MuteQuack implements QuackBehavior {
    public void quack() {
        System.out.println("<< Silence >>");
    }
}

public class Squeak implements QuackBehavior {
    public void quack() {
        System.out.println("Squeak");
    }
}
```

- ④ **Type and compile the test class (MiniDuckSimulator.java).**

```
public class MiniDuckSimulator {
    public static void main(String[] args) {
        Duck mallard = new MallardDuck();
        mallard.performQuack();
        mallard.performFly();
    }
}
```

This calls the MallardDuck's inherited performQuack() method, which then delegates to the object's QuackBehavior (i.e., calls quack() on the duck's inherited quackBehavior reference).

Then we do the same thing with MallardDuck's inherited performFly() method.

- ⑤ **Run the code!**

```
File Edit Window Help Yadayadayada
%java MiniDuckSimulator
Quack
I'm flying!!
```

Setting behavior dynamically

What a shame to have all this dynamic talent built into our ducks and not be using it! Imagine you want to set the duck's behavior type through a setter method on the Duck class, rather than by instantiating it in the duck's constructor.

1 Add two new methods to the Duck class:

```
public void setFlyBehavior(FlyBehavior fb) {
    flyBehavior = fb;
}

public void setQuackBehavior(QuackBehavior qb) {
    quackBehavior = qb;
}
```

We can call these methods anytime we want to change the behavior of a duck on the fly.

Editor note: gratuitous pun – fix

Duck
FlyBehavior flyBehavior QuackBehavior quackBehavior
swim() display() performQuack() performFly() setFlyBehavior() setQuackBehavior() // OTHER duck-like methods...

2 Make a new Duck type (ModelDuck.java).

```
public class ModelDuck extends Duck {
    public ModelDuck() {
        flyBehavior = new FlyNoWay();
        quackBehavior = new Quack();
    }

    public void display() {
        System.out.println("I'm a model duck");
    }
}
```

Our model duck begins life grounded... without a way to fly.

3 Make a new FlyBehavior type (FlyRocketPowered.java).

```
public class FlyRocketPowered implements FlyBehavior {
    public void fly() {
        System.out.println("I'm flying with a rocket!");
    }
}
```

That's okay, we're creating a rocket-powered flying behavior.



4 Change the test class (MiniDuckSimulator.java), add the ModelDuck, and make the ModelDuck rocket-enabled.

```
public class MiniDuckSimulator {
    public static void main(String[] args) {
        Duck mallard = new MallardDuck();
        mallard.performQuack();
        mallard.performFly();

        Duck model = new ModelDuck();
        model.performFly();
        model.setFlyBehavior(new FlyRocketPowered());
        model.performFly();
    }
}
```

If it worked, the model duck dynamically changed its flying behavior! You can't do THAT if the implementation lives inside the Duck class.

5 Run it!

```
File Edit Window Help Yabbadabbadoo
%java MiniDuckSimulator

Quack

I'm flying!!

I can't fly

I'm flying with a rocket!
```



The first call to `performFly()` delegates to the `flyBehavior` object set in the `ModelDuck`'s constructor, which is a `FlyNoWay` instance.

This invokes the model's inherited behavior setter method, and...voilà! The model suddenly has rocket-powered flying capability!



To change a duck's behavior at runtime, just call the duck's setter method for that behavior.

The Big Picture on encapsulated behaviors

Okay, now that we've done the deep dive on the duck simulator design, it's time to come back up for air and take a look at the big picture.

Below is the entire reworked class structure. We have everything you'd expect: ducks extending Duck, fly behaviors implementing FlyBehavior, and quack behaviors implementing QuackBehavior.

Notice also that we've started to describe things a little differently. Instead of thinking of the duck behaviors as a *set of behaviors*, we'll start thinking of them as a *family of algorithms*. Think about it: in the SimUDuck design, the algorithms represent things a duck would do (different ways of quacking or flying), but we could just as easily use the same techniques for a set of classes that implement the ways to compute state sales tax by different states.

Pay careful attention to the *relationships* between the classes. In fact, grab your pen and write the appropriate relationship (IS-A, HAS-A, and IMPLEMENTS) on each arrow in the class diagram.

Make sure you do this.

